

VERITAS: Trajectory-Level Authorization for Concurrent AI Agents

Crizan Belem Ribeiro

Independent Researcher, São Paulo, Brazil

belemcrizan@gmail.com ORCID: 0009-0004-8920-7135

<https://github.com/belemcrizan/Veritas> Artifact version: VERITAS v0.1.0

Technical report v0.1 (working paper) — August 2026

Describes the current prototype; comparative and large-scale results are future work

Abstract

Autonomous agents increasingly invoke consequential tools — payments, database mutations, outbound messaging — through authorization layers that evaluate each tool call in isolation. Per-call authorization is structurally blind to a class of violations that arise only at the level of *trajectories*: fractionated actions that individually satisfy a threshold, concurrent agents that jointly exhaust a shared budget, time-of-check/time-of-use races against changing state or policy, and post-approval argument mutation. We present VERITAS, an execution boundary that authorizes trajectories rather than calls. VERITAS models authorization budgets as a resource algebra over an ordered monoid, issues *consumable capabilities* — short-lived, signed, single-use credentials bound to the state and policy version under which they were verified — and preserves global invariants across concurrent agents through three coordination mechanisms: linearizable compare-and-swap, pre-allocated partitioning, and a hybrid. The target design keeps SMT solving outside the runtime path; the current prototype includes a separately authored bounded SMT-LIB model, while runtime authorization uses compiled JSON policy tables, integer arithmetic, and signature verification. In the current deterministic prototype, VERITAS blocks all eleven implemented attack scenarios of VERITAS-Bench. In a 40-request concurrent SQLite experiment against a 10,000-unit budget, 33 reservations of 300 succeed, total reserved consumption is limited to 9,900, and no overspend is observed. Comparative baselines, large-scale randomized concurrency tests, and the measurement of the coordination–autonomy trade-off that this design exposes remain future work and are not claimed here.

1 Introduction

Consider a finance agent authorized to transfer up to 10,000 units per call. It receives an injected instruction and requests twelve transfers of 900 to the same destination within twenty minutes. Each request is below threshold; a stateless per-call rule that checks only *amount* $\leq$ 10,000 admits all twelve. The total is 10,800. Session-scoped counters can catch this pattern within one session; they do not catch it across sessions or across agents.

This is not a corner case. It is one instance of a general gap: *agent safety is a property of trajectories under concurrency, not of individual tool calls*. Four failure modes

follow directly from per-call evaluation:

- (i) **Composition.** A sequence of individually admissible actions violates an aggregate invariant (fractionation, cross-tool data exfiltration).
- (ii) **Concurrency.** Two agents read the same remaining budget and both succeed — a double-spend of authority.
- (iii) **Staleness.** An action is verified against state or policy that has changed by the time it executes (TOCTOU, policy race).
- (iv) **Mutation.** A human approves one request and the agent executes a different one.

Prior work addresses pieces of this gap. Session call counters in Cedar-for-agents handle one-agent, one-session counting. Delegation-scope compression prevents privilege *expansion* along a chain but not aggregate *exhaustion* across a population. Ephemeral-credential gateways remove standing privilege but still authorize each call against a snapshot. In the systems and public artifacts reviewed so far, we did not identify an evaluation that jointly measures composition, concurrent residual exhaustion, and the coordination–autonomy trade-off.

Contributions.

1. A formalization of trajectory admissibility over a *resource algebra*: budgets as elements of an ordered monoid, actions as consumption vectors, and three classes of invariants (resource, temporal, separation) with explicit trust assumptions (§2).
2. *Consumable capabilities*: a Prepare–Verify–Commit protocol that issues single-use, signed credentials carrying a residual budget and bound to a hash of declared state preconditions and to a policy version, with a compensation path for failed commits (§3).
3. Three coordination mechanisms for preserving global invariants across concurrent agents, and a measurement plan for the *coordination–autonomy trade-off* between them (§6).
4. VERITAS-Bench, a reproducible benchmark harness of eleven adversarial families, intended for future open release (§5).

We make no claim of production readiness. VERITAS is a research instrument whose purpose is to make the trade-off measurable.

2 Threat Model and Problem Statement

2.1 Setting

An *agent* is an untrusted process that emits requests to execute actions on *protected tools*. Requests are normalized into an *Agent Safety Intermediate Representation* (ASIR): a canonical record of principal, delegation chain, action, resource, parameters, purpose, sensitivity labels, and provenance of the observations that motivated the request. Canonical serialization yields a stable hash $h(\alpha)$ for each ASIR α .

2.2 Adversary

The adversary controls one or more agents. It may submit arbitrary ASIRs, replay or parallelize requests, coordinate across agents, delay execution, and alter parameters after a human has seen a rendering of the request. It cannot forge signatures, compromise the capability issuer’s key, or desynchronize clocks beyond a bound δ . Compromised orchestrators and injected observations are in scope; a compromised issuer is not.

2.3 Resource algebra

Let $(M, +, 0, \leq)$ be an ordered commutative monoid. A *budget* is a vector $R \in M^k$ indexed by resource dimensions (e.g., money per destination per window, API calls per hour, expected loss). Each action a has a consumption vector $c(a) \in M^k$. A trajectory $\tau = (a_1, \dots, a_n)$ induces $R_0 \xrightarrow{c(a_1)} R_1 \rightarrow \dots \rightarrow R_n$ with $R_t = R_{t-1} - c(a_t)$.

Definition 1 (Admissible trajectory). *Given a safe region $S \subseteq M^k$ (a hyper-rectangle in this work), τ is admissible iff $R_t \in S$ for all $t \in \{0, \dots, n\}$.*

Only dimensions that satisfy the monoid axioms enter the algebra. Probability-valued risk does not (it is not additive); delegation depth does not (it is structural). Expected loss $\sum_i p_i L_i$ does.

2.4 Invariant classes

Class I (resource). $\sum_{a \in \tau|_w} c_j(a) \leq B_j$ over a window w and grouping key (destination, session, principal).

Class II (temporal). Ordering constraints expressed as finite automata per (session, resource): *Approve* $\prec$ *Transfer*; *SensitiveRead* $\Rightarrow \neg$ *ExternalSend*.

Class III (separation). Relational constraints: *initiator* $\neq$ *approver*; *depth*(*delegation*) $\leq d$.

2.5 Properties

Definition 2 (No Double-Spend). *Let $\text{Committed}_r(t)$ be the consumption of actions on r that have committed within the active window, and $\text{Prepared}_r(t)$ the consumption held by live reservations (prepared, not yet committed, not expired or compensated). Then for every resource r and time t ,*

$$\text{Committed}_r(t) + \text{Prepared}_r(t) \leq B_r.$$

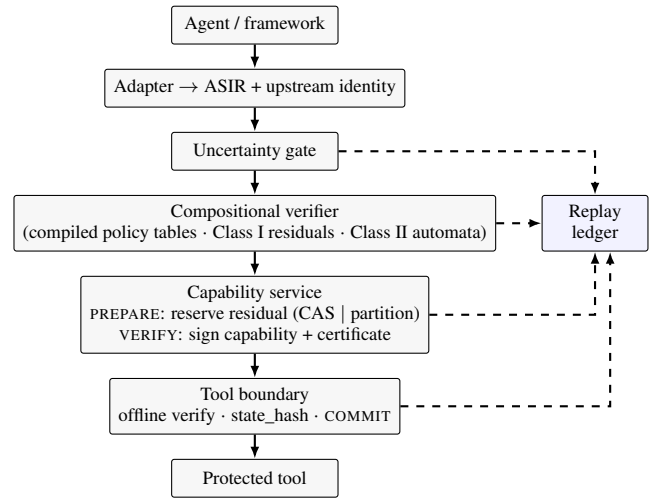


Figure 1: Target execution path; Table 1 identifies the components implemented in Cycle 1. The solver is absent from this path.

Under partitioning, with $P_{r,g}$ the share pre-allocated to agent g and U_r the unpartitioned central residual, the issuer maintains $\sum_g P_{r,g} + U_r \leq B_r$ at every rebalancing step.

Note that summing the residual *snapshots* carried by live capabilities is not the right quantity: two valid capabilities may carry snapshots taken at different times. The property is about reservations and commits, not about token contents.

Definition 3 (Trajectory Safety). *If the modeled protocol authorizes τ , then τ satisfies every invariant represented in the model.*

Trajectory Safety is stated about the *modeled protocol* — compiled tables, residual reservations, the commit protocol, and the coordination mechanism — not about the specification alone, which would make it vacuous. We argue it under the following assumptions, several of which induce attack scenarios in VERITAS-Bench, while the remainder are explicit trust assumptions.

Assumption 1 (H1–H6). (H1) the issuer is honest and its signing key is uncompromised; (H2) signatures are unforgeable; (H3) every protected tool verifies the capability certificate before executing; (H4) clock skew between issuer and tool is bounded by δ ; (H5) upstream identity and delegation tokens are authentic; (H6) no capability survives its commit, compensation, or expiry.

We also require a *progress* property — admissible trajectories are authorized with probability close to one — which we treat as an empirical claim and measure rather than prove, because the uncertainty gate (§3.5) is stochastic.

3 Design

3.1 Compile-time verification, run-time tables

Target design. In the intended system, policies are written in Cedar for per-call (Class III partial) rules and in a small invariant DSL for Classes I and II. A compiler emits two artifacts: (a) SMT-LIB formulas, checked in continuous

integration for inter-policy consistency, scope containment ($scope(cap) \subseteq allow(policy)$), and bounded unreachability of forbidden states (depth k); and (b) immutable decision tables plus residual parameters for the runtime. A satisfiable query $\exists \tau. Authorized(\tau) \wedge Violation(\tau)$ produces a counterexample attached to the pull request; unsatisfiability within the bound is formal evidence for the model. The runtime never invokes the solver. In the current prototype the executable policy is reviewed JSON compiled to tables, and the bounded SMT-LIB model is authored separately (Table 1).

3.2 Consumable capabilities

A capability κ is a signed record

$$\kappa = \langle h(\alpha), \sigma, \rho, v, \nu, i, t_{\text{exp}}, \pi \rangle_{\text{sk}}$$

where σ is the hash of declared state preconditions, ρ the residual vector after this action, v the policy version, ν a nonce, i the chain index, t_{exp} a short expiry, and π a compact certificate of scope containment. Consumption produces a successor with $\rho' \leq \rho$ and $i' = i + 1$; a consumed capability is never reusable. Chaining alone does *not* prevent double-spend — two agents may read the same ρ — which is why reservation is delegated to a coordination mechanism (§3.4).

3.3 Prepare–Verify–Commit

1. **Prepare.** Reserve $c(a)$ against the residual via the coordination mechanism; log intent.
2. **Verify.** Evaluate compiled tables; compute σ over declared preconditions; sign κ .
3. **Commit.** The tool boundary verifies signature, expiry, nonce freshness, policy version, and recomputes σ from current state. Mismatch yields `STALE_CAPABILITY`; at most $k=3$ re-verifications precede escalation to a human. On success the tool executes and emits an idempotent acknowledgment.
4. **Compensate.** Absent acknowledgment by T_{ack} , the service queries the tool by idempotency key; the residual is restored only on confirmed non-execution, otherwise the loss is recorded.

Human approvals are signatures over $h(\alpha)$ with nonce and expiry; the rendering shown to the approver is derived deterministically from the same α , closing the *what-you-see-is-what-you-sign* gap.

3.4 Coordination mechanisms

CAS serializes every reservation on a per-resource key with a linearizable conditional write. It is exact and simple, and it concentrates contention on a single point per resource. **Partition** splits the budget across agents before execution, requiring no runtime coordination but introducing fragmentation: an agent may be denied an action that is feasible in aggregate. **Hybrid** partitions by default and spills into a CAS-managed remainder; rebalancing is asynchronous and off the hot path. The choice among these is not a design detail — it is the object of study.

Algorithm 1: Reservation under the three mechanisms

Input: resource r , cost c , agent g

```

1 CAS( $r, c$ ):
2   repeat
3      $\rho \leftarrow \text{read}(r)$ ;
4     if  $\rho - c \notin S$  then return DENY;
5     success  $\leftarrow \text{compare\_and\_swap}(r, \rho, \rho - c)$ ;
6   until success;
7   return ALLOW;
8 PARTITION( $r, c, g$ ):
9    $\rho_g \leftarrow \text{local\_share}(r, g)$ ;    // allocated before
    execution
10  if  $\rho_g - c \notin S$  then return DENY;
11   $\rho_g \leftarrow \rho_g - c$ ; return ALLOW;
12 HYBRID( $r, c, g$ ):
13  if PARTITION( $r, c, g$ ) = ALLOW then return ALLOW;
14  return CAS( $r_{\text{global}}, c$ );           // spill into
    unpartitioned remainder

```

3.5 Uncertainty gate

A split-conformal gate computes per-field prediction sets over the ASIR from model log-probabilities, with miscov-
erage α chosen per risk class. A singleton set proceeds; a larger set escalates only the ambiguous field. The gate emits a statistical guarantee only when the calibration mass for the action type is at least $1/\alpha$; high-risk actions ($\alpha < 10^{-2}$) never depend on it and are decided deterministically or by a human. In the present cycle the gate is an audited bypass in the hot path and is evaluated separately.

3.6 Verifiable replay ledger

Every observation, ASIR, gate decision, verifier decision, capability, approval, tool input/output, commit acknowledgment, and compensation is a node $H_i = \text{Hash}(H_{\text{parents}} \parallel \text{payload}_i)$ in an append-only DAG. The ledger supports *trace replay* (exact reproduction, no model call), *tool replay* (recorded tool outputs as mocks), and *intervention replay* (substitute one node, propagate). We claim only that, within the captured re-executable environment, an intervention changes the trajectory in the observed way; we make no general causal claim.

3.7 Safety argument

Proposition 1 (Trajectory Safety in the abstract transition model). *Under H1–H6 and with any of the three mechanisms, every trajectory authorized by the modeled protocol satisfies all supported Class I invariants; with the session automaton enabled, all Class II invariants; and all Class III invariants encoded in the compiled tables.*

This is a safety argument for the modeled protocol, not a refinement proof of the Python/SQLite implementation. The bounded SMT-LIB model in the repository checks the Class I fragment up to depth k ; connecting the DSL, the compiled tables, the SQLite transaction, the capability lifecycle, and the boundary checks into one mechanized refinement is future work.

Proof sketch. Class I: by H1–H2 only the issuer produces valid κ ; for the CAS mechanism, the linearizable reservation

transaction preserves $Committed_r(t) + Prepared_r(t) \leq B_r$ at every successful transition; for partitioning, disjoint pre-allocated shares and the unpartitioned remainder satisfy $\sum_g P_{r,g}(t) + U_r(t) \leq B_r$. Hence every successful reservation is backed by non-overlapping authority, and cumulative prepared and committed consumption cannot exceed the policy budget (No Double-Spend). By H3 no tool executes without a valid κ ; by H6 and the nonce store no κ executes twice; by H4 expiry is meaningful. Hence every executed action was reserved, and the reserved sequence stays within S . Class II: the automaton transition is evaluated in Verify against the ledger-recorded session state, which by the hash chain is tamper-evident. Class III: compiled tables are total over the ASIR schema and fail closed. The compile-time scope-containment check (bounded) ensures no table admits an action outside $allow(policy)$ up to depth k . $\square$

The bound k and the cooperative-tool assumption H3 are the two places where the argument is weaker than we would like; §7 discusses both.

4 Implementation: design target vs. current prototype

Python 3.12; no external services in the reference configuration. Cloud SDKs are confined to an `adapters/` package enforced by a lint rule. Everything runs with `veritas demo`, `veritas bench`, and `veritas perf`. Source code, policies, the SMT-LIB bounded model, and the benchmark harness are available at <https://github.com/belemcrizan/Veritas>. Table 1 states, component by component, where the prototype stands relative to the design of §3; every claim in this report should be read against it.

5 VERITAS-Bench

Eleven families over three protected tools (payments, database, messaging). Several scenarios are currently sequential rather than multi-agent, and some model a representative slice of the family rather than its full sequence (e.g., *compensation abuse* tests idempotent duplicate compensation but not the full timeout $\rightarrow$ status query $\rightarrow$ late-ack $\rightarrow$ indeterminate-loss path; *temporal evasion* uses an altered request timestamp rather than spreading across windows). The families are: *atomic* (single over-threshold call), *fractionation*, *temporal evasion* (spreading across windows), *parallel double-spend*, *delegation laundering*, *approval mutation*, *cross-tool composition* (read PII then send externally), *policy race*, *clock skew*, *capability replay*, and *compensation abuse*.

Planned baselines, all to receive exactly the same event stream: (B0) no protection; (B1) stateless per-call threshold filter; (B2) per-call Cedar semantics with session call counters; (B3) an OPA-equivalent rule set; optionally (B4) attenuated OAuth tokens with delegation-scope compression. Only B0 exists in the current harness. **No comparative coverage or performance claim is made in this draft.** Scenarios ship with the repository (`src/veritas/bench.py`).

6 Evaluation: current results and measurement plan

We separate what has been measured from what the design is built to measure.

6.1 Current results

Adversarial scenarios. In the deterministic harness, VERITAS blocks 11 of 11 implemented scenarios. Because no baseline other than B0 is implemented, this is a regression result for VERITAS, not a comparison.

Concurrent reservation. Forty concurrent requests of 300 units are issued against a 10,000-unit budget through the SQLite CAS store. Thirty-three reservations succeed, total reserved consumption is 9,900, and no overspend is observed. This is evidence that the reference store serializes correctly; it is not evidence at the scale (10^6 interleavings, up to 1,000 agents) the design calls for.

Local microbenchmarks. `veritas perf` reports policy-table lookup and Ed25519 verification latency on the host where it runs; figures are machine-dependent and are therefore not tabulated here. The design targets are $p_{95} < 5$ ms for the policy path and < 1 ms for boundary verification.

6.2 Measurement plan

Four questions remain open and define the next cycle.

Q1 — Coverage. Run B1–B4 in the same harness and report, per family, which systems admit it. Hypotheses: B1 admits everything except *atomic*; B2 additionally blocks single-session fractionation; B4 additionally blocks delegation laundering; none blocks parallel double-spend, policy race, approval mutation, or cross-tool composition. These are hypotheses, not results.

Q2 — No Double-Spend at scale. Property-based generation of interleavings for $g \in \{2, 10, 100, 1000\}$ with randomized timing, injected partitions, and dropped acknowledgments, until 10^6 attempts per mechanism; a naïve read-then-write gateway is included so that the test demonstrably detects violations. Raw data will be preserved in the repository.

Q3 — The coordination–autonomy trade-off. For each mechanism and concurrency level, measure throughput, $p_{50}/p_{95}/p_{99}$, and the *feasible-denial rate* (actions denied although the aggregate budget could accommodate them). The store must exhibit genuine contention (Redis WATCH/MULTI or Postgres SELECT ...FOR UPDATE with injected latency); a single SQLite lock serializes all three mechanisms identically and would hide the curve. The hypothesized shape is that CAS throughput degrades with contention at zero feasible denial, partitioning sustains throughput at rising feasible denial, and the hybrid interpolates. We conjecture, but do not prove, that for any mechanism preserving No Double-Spend, throughput $\times (1 - \text{feasible_denial})$ is bounded above by a function of the store’s linearization latency. If measured, this curve — not the system — is the result.

Table 1: Design target versus current prototype (cycle 1).

Design target	Current prototype
Cedar compiled to tables + SMT by one compiler	Executable policy is reviewed JSON compiled to immutable tables; Cedar is a documentation sketch; SMT-LIB model is authored separately
PASETO v4.public capabilities	Ed25519 envelope <code>veritas.v1</code> , cryptographically checked; not PASETO-compliant
Successor capability chain	Chain fields present; successor issuance not yet implemented
Randomized property-based concurrency tests	Deterministic tests (12), including a 40-request concurrent SQLite run
Redis/Postgres contention experiments	SQLite <code>BEGIN IMMEDIATE</code> only
Trace, tool, and intervention replay	Trace and intervention replay; tool replay partial
Mechanized refinement proof	Safety argument + bounded SMT model
Calibrated shift detector in the gate	Audited bypass + functional split-conformal categorical gate; no shift detector
OIDC/SPIFFE signature validation	Upstream identity accepted as structured input; not validated

Q4 — Overhead. Marginal cost of trajectory-level checking over a per-call table (B1/B2), reported as $p_{50}/p_{95}/p_{99}$ on a fixed reference host.

7 Limitations

- **Cooperative tools (H3).** State-bound authorization requires the tool to recompute σ . Non-cooperative tools degrade to short expiry behind a proxy — which is what existing gateways already do.
- **Bounded verification.** The SMT check covers a model of the policy up to depth k , not the Python implementation. Property-based tests narrow, but do not close, that gap.
- **Partition fragmentation.** Feasible denial is a real cost, not an artifact; rebalancing reduces but does not eliminate it.
- **Gate under shift.** Conformal coverage is guaranteed under exchangeability; we measure, and do not promise, coverage under injection.
- **Baselines.** Only B0 exists; commercial gateways cannot be reproduced, and planned baselines reproduce published semantics, not products.
- **Prior art and release.** A patent and literature review is pending; no novelty claim should be read into this draft. The repository is under a provisional license until that review concludes; the intended open-release subset is the ASIR schema, the invariant DSL, adapters, and the benchmark scenarios.

8 Related Work

Agent authorization. Cedar [1] provides analyzable per-call policies with a verified validator and SMT encoding; agent-oriented interventions add per-session call counters. Multi-hop delegation with scope compression prevents privilege expansion along a chain [2]. Ephemeral-credential gateways remove standing secrets and bind tokens to a human sponsor. In the artifacts reviewed, requests are evaluated individually against a snapshot.

Capabilities and proof-carrying authorization. Object-capability systems bind authority to unforgeable references; proof-carrying code [3] attaches a checkable certificate to an artifact. VERITAS combines both with consumption and state binding.

Concurrency control. Linearizability [4] and consensus [5] supply the exactness we require of CAS; UTXO models supply single-spend by ledger serialization, which we adapt with an issuer in the ledger’s role.

Uncertainty. Split conformal prediction [6], its non-exchangeable extensions [7], and conformal risk control [8] motivate the gate.

Standards. NIST’s 2026 agent identity and authorization work [9] and the OWASP agentic top ten [10] name identification, authorization, auditing, and non-repudiation as open needs; we did not find treatment of aggregate invariants across concurrent agents in the documents reviewed.

9 Conclusion

Per-call authorization cannot see the violations that matter most for autonomous agents: those that emerge from composition and concurrency. VERITAS moves the unit of authorization to the trajectory, makes the budget an algebraic object, makes the credential consumable and state-bound, and makes the cost of global safety measurable. The current prototype shows the mechanism works at small scale; the coordination–autonomy frontier described in §6, once measured, is the intended result — the system is the instrument.

References

- [1] J. W. Cutler et al. Cedar: A new language for expressive, fast, safe, and analyzable authorization. *Proc. ACM Program. Lang.* (OOPSLA), 2024.
- [2] Amazon Web Services. Enforce least-privilege authorization in multi-agent AI chains using Cedar. AWS Security Blog, July 2026.
- [3] G. C. Necula. Proof-carrying code. *POPL*, 1997.
- [4] M. Herlihy and J. Wing. Linearizability: A correctness condition for concurrent objects. *ACM TOPLAS*, 1990.
- [5] D. Ongaro and J. Ousterhout. In search of an understandable consensus algorithm. *USENIX ATC*, 2014.
- [6] V. Vovk, A. Gammelman, and G. Shafer. *Algorithmic Learning in a Random World*. Springer, 2005.
- [7] R. F. Barber et al. Conformal prediction beyond exchangeability. *Ann. Statist.*, 2023.
- [8] A. N. Angelopoulos et al. Conformal risk control. *ICLR*, 2024.

- [9] NIST. AI Agent Standards Initiative; Concept paper on software and AI agent identity and authorization, 2026.
- [10] OWASP. Top 10 for Agentic Applications, 2026.